CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

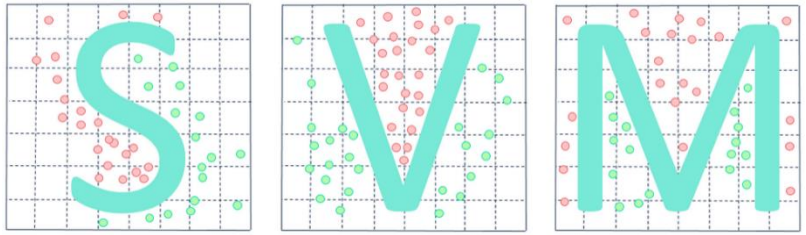
Bacharelado em Sistemas da Informação

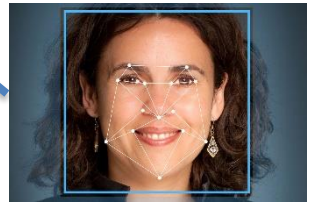
Prof. Marco André Abud Kappel

Aula 7 – *Support Vector Machines (SVM)*

Support Vector Machines (SVM)

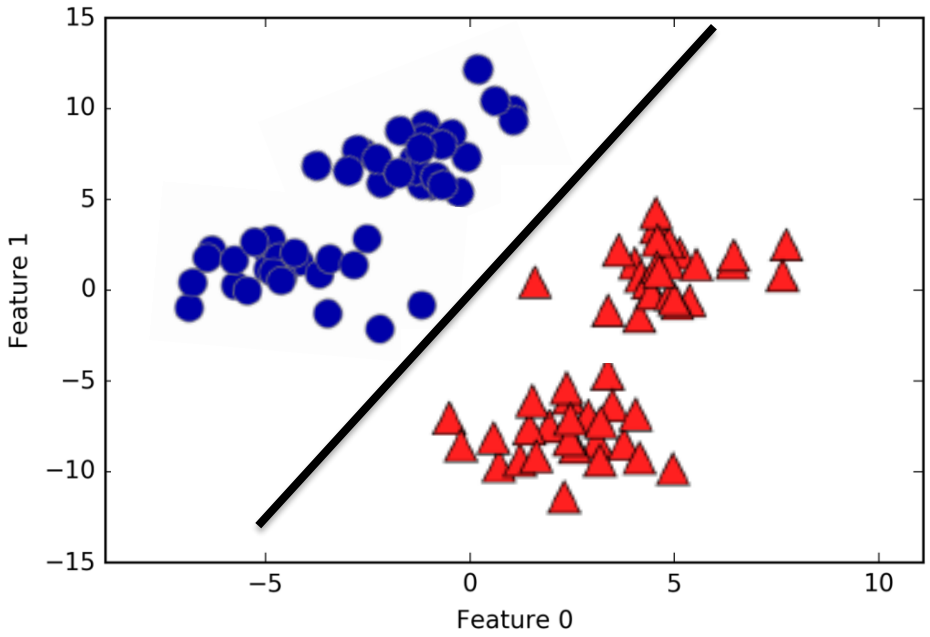
- **Introdução**
  - Em geral, possui **bons resultados** em relação aos outros algoritmos de aprendizagem de máquina.
  - Muito utilizado em **tarefas complexas** como:
    - Reconhecimento de caracteres
    - Reconhecimento de voz
    - Reconhecimento de imagens





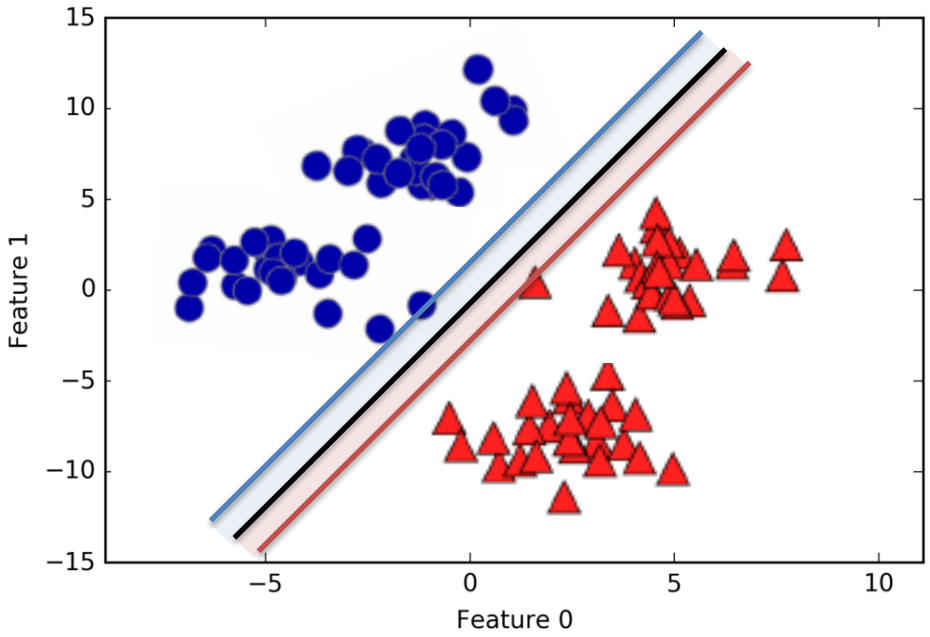
Support Vector Machines (SVM)

- **Funcionamento**
  - O SVM padrão toma como entrada um conjunto de dados e encontra uma **linha de separação**, mais comumente chamada de "hiperplano" entre dados de duas classes.



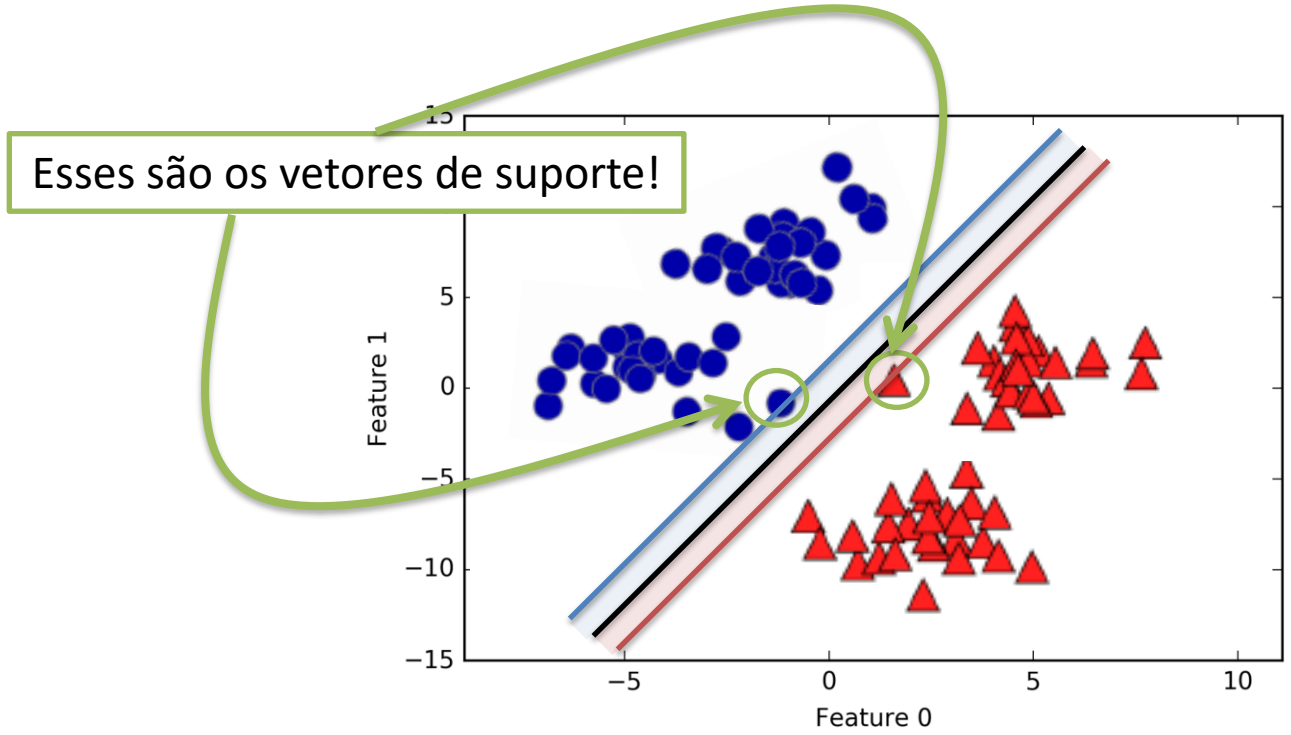
Support Vector Machines (SVM)

- **Funcionamento**
  - Para isso, ele identifica os **vetores** que possibilitam uma separação com **margem máxima** entre as classes.



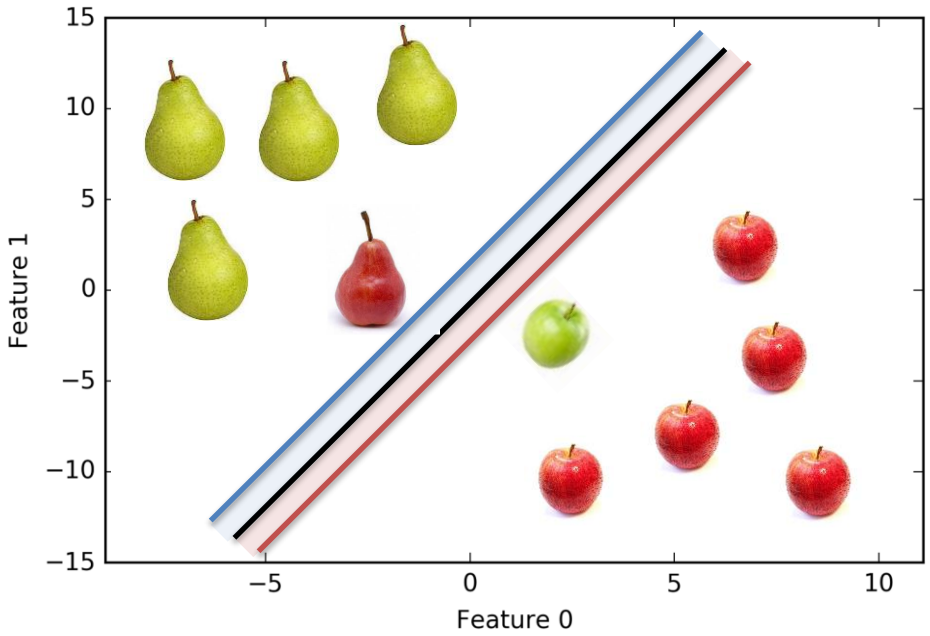
Support Vector Machines (SVM)

- **Funcionamento**
  - Para isso, ele identifica os **vetores** que possibilitam uma separação com **margem máxima** entre as classes.



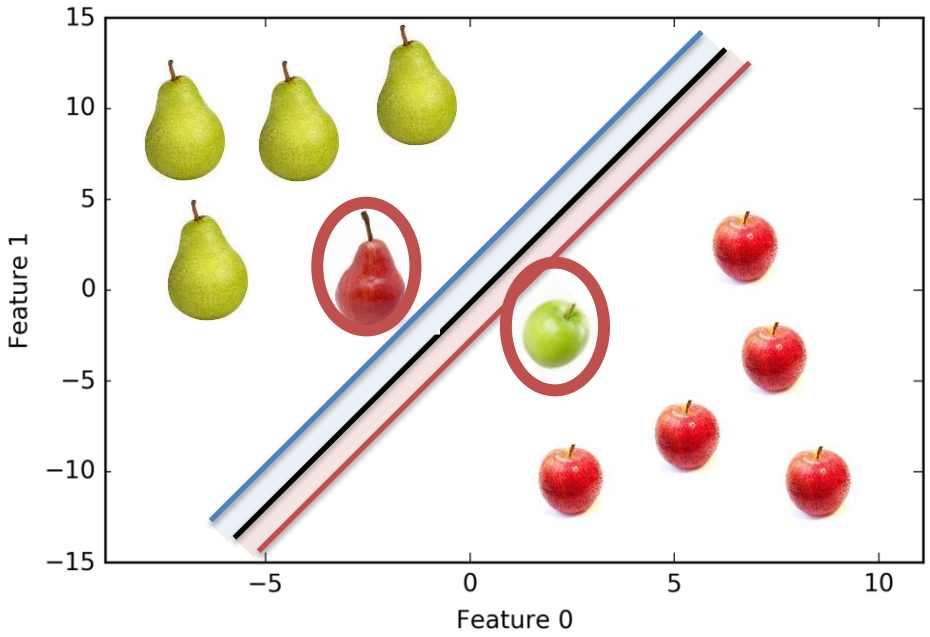
Support Vector Machines (SVM)

- **Funcionamento**
  - Os **vetores de suporte** serão os elementos de cada classe que **mais se distanciam** em relação aos elementos mais frequentes dela.



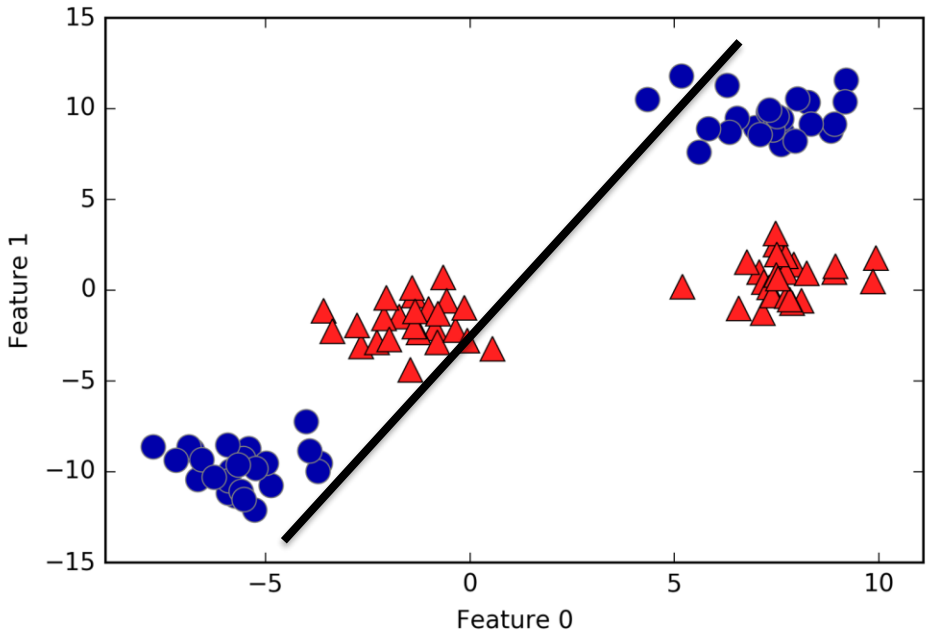
Support Vector Machines (SVM)

- **Funcionamento**
  - A fase de **treinamento** do algoritmo é, basicamente, **identificar** quais são os vetores de suporte, que possibilitam a construção do **hiperplano**.



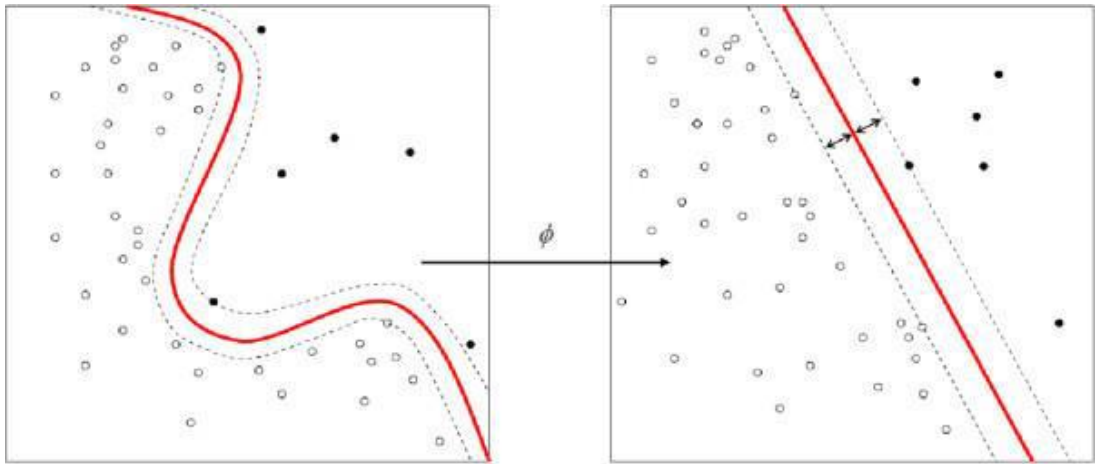
Support Vector Machines (SVM)

- **Funcionamento**
  - No **mundo real**, a maior parte dos problemas não se comporta de maneira **linear**.
  - O que fazer quando encontramos uma situação como essa?



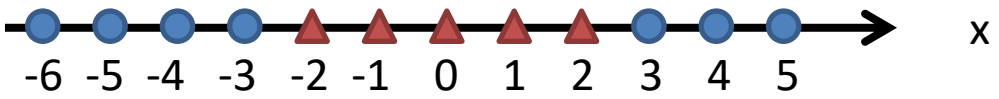
Support Vector Machines (SVM)

- **Kernelized Support Vector Machines**
  - Em problemas **não lineares**, é utilizada uma técnica chamada "**Kernel trick**".
  - Esse truque, basicamente, **transforma** o conjunto de dados de maneira que o problema se torne **linear**.



Support Vector Machines (SVM)

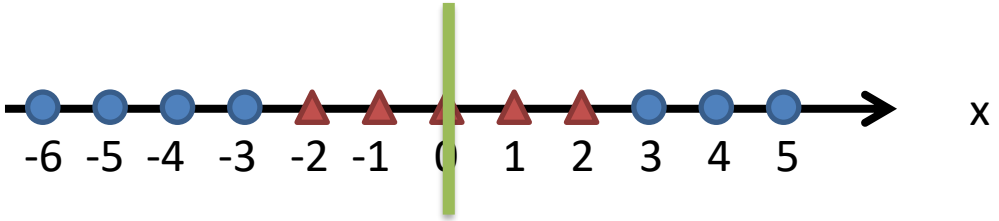
- **Kernelized Support Vector Machines**
  - Como um exemplo simples, considere que queremos classificar o conjunto de dados **unidimensional** abaixo:



  - Em qual valor poderíamos fazer uma **separação linear** nos dados, de forma que a **classificação** ficasse correta?

Support Vector Machines (SVM)

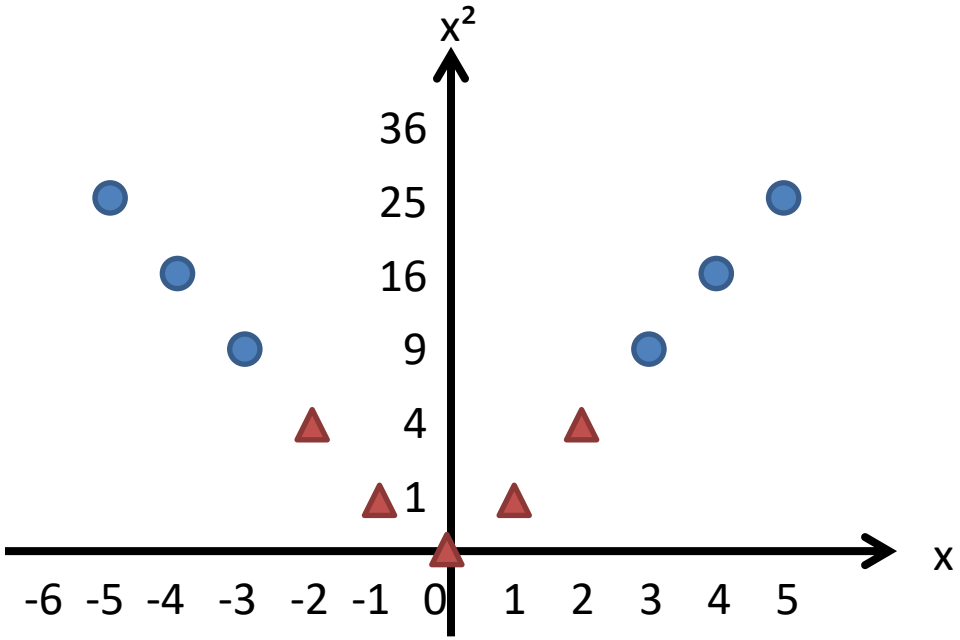
- **Kernelized Support Vector Machines**
  - Como um exemplo simples, considere que queremos classificar o conjunto de dados **unidimensional** abaixo:



  - Em qual valor poderíamos fazer uma **separação linear** nos dados, de forma que a **classificação** ficasse correta?

Support Vector Machines (SVM)

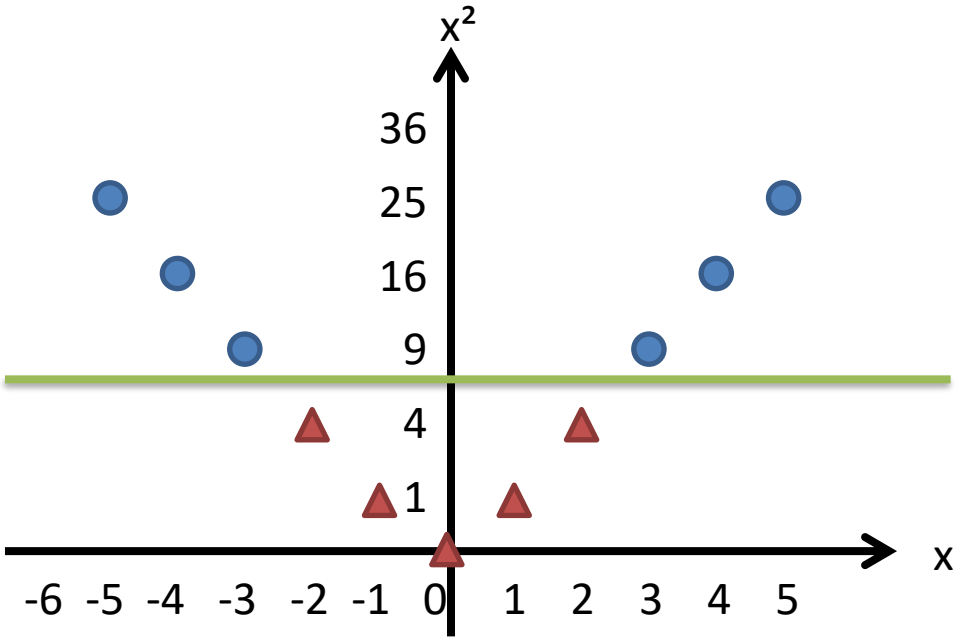
- **Kernelized Support Vector Machines**
  - A transformação, neste caso, poderia ser algo simples, como adicionar uma **nova dimensão** ao problema.
  - Esta **nova dimensão** corresponderia ao valor da idade **elevado ao quadrado**.



- E agora, seria possível fazer uma separação linear dos dados?

Support Vector Machines (SVM)

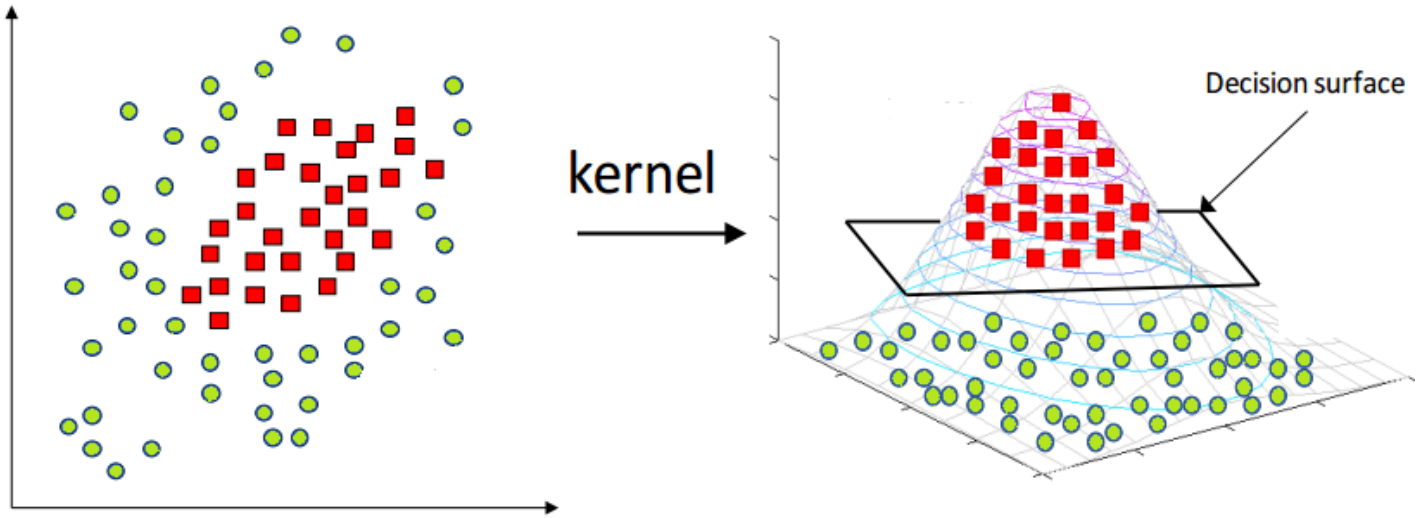
- **Kernelized Support Vector Machines**
  - A transformação, neste caso, poderia ser algo simples, como adicionar uma **nova dimensão** ao problema.
  - Esta **nova dimensão** corresponderia ao valor da idade **elevado ao quadrado**.



- E agora, seria possível fazer uma separação linear dos dados? **Sim!**

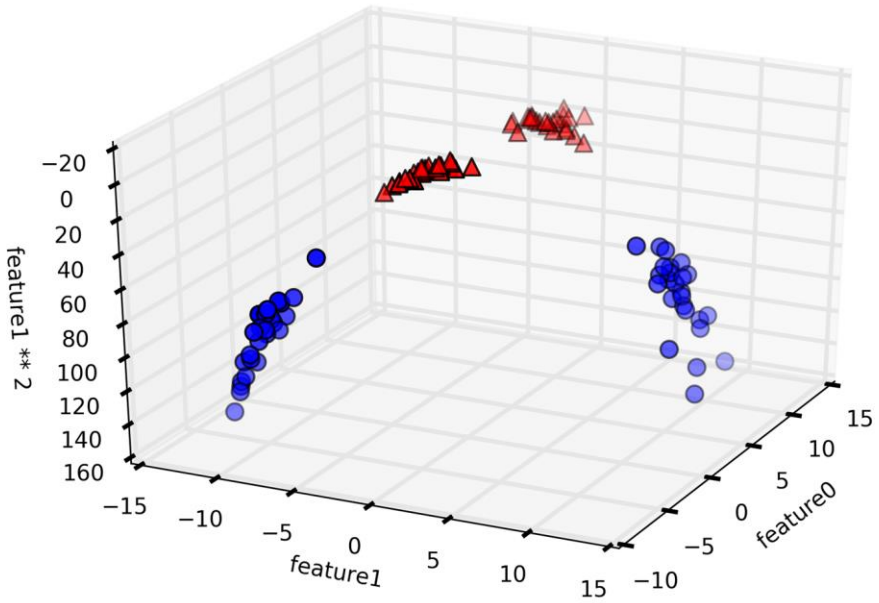
Support Vector Machines (SVM)

- **Kernelized Support Vector Machines**
  - A transformação envolve, por exemplo, adicionar mais uma **dimensão** aos dados do problema.
  - Com isso, os dados podem se tornar **separáveis** por um hiperplano.



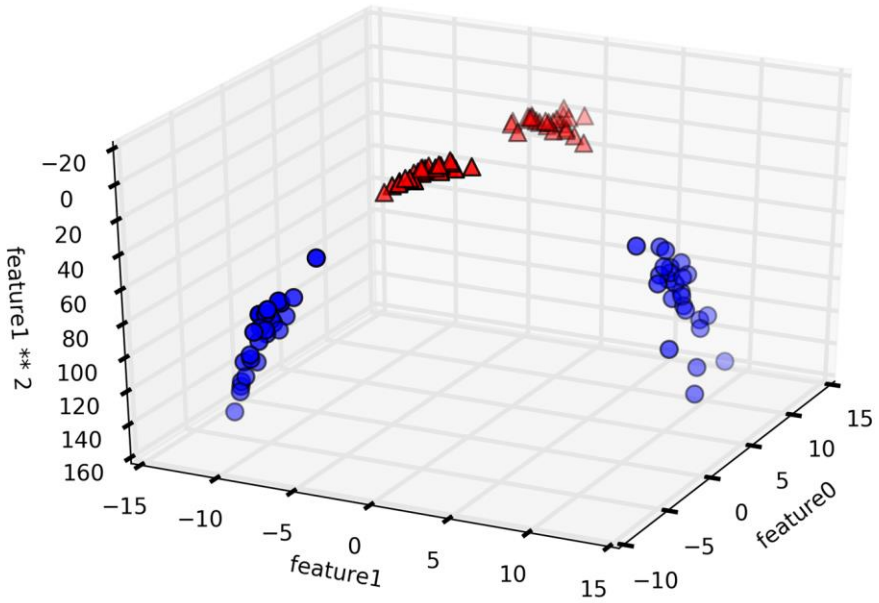
Support Vector Machines (SVM)

- **Funcionamento**
  - Neste caso, poderíamos **transformar** cada ponto (x, y) em um ponto (x, y, y^2).
  - O gráfico, agora com **três dimensões**, ficaria assim:



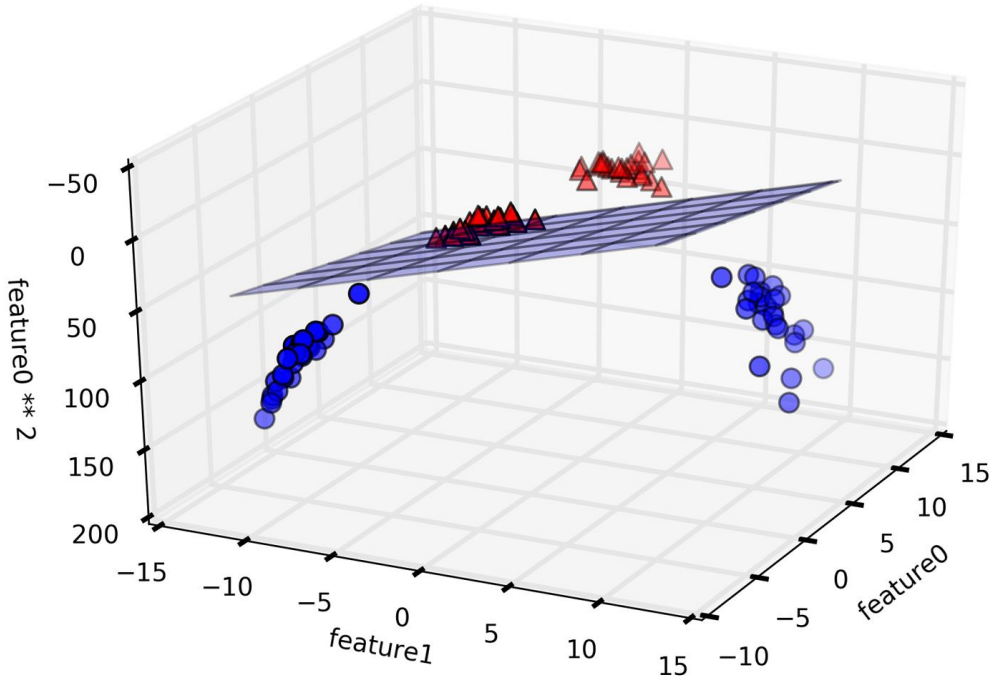
Support Vector Machines (SVM)

- **Funcionamento**
  - Neste caso, poderíamos **transformar** cada ponto (x, y) em um ponto (x, y, y^2).
  - O gráfico, agora com **três dimensões**, ficaria assim:



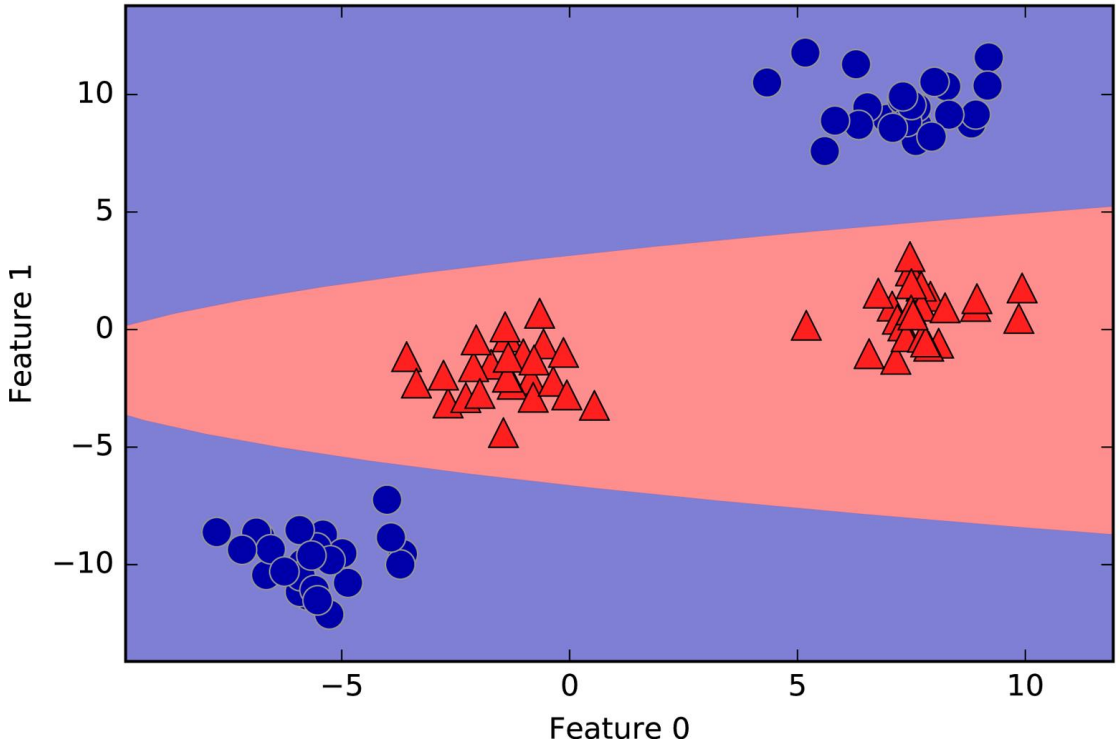
Support Vector Machines (SVM)

- **Funcionamento**
  - Neste novo conjunto de dados, é possível traçar um **hiperplano** separando as classes!



Support Vector Machines (SVM)

- **Funcionamento**
  - Se voltarmos para a visualização dos dados **originais**, é possível perceber que o SVM deixou de fazer uma simples **separação linear**.



Support Vector Machines (SVM)

- **Funcionamento**
  - Existem diferentes **tipos** de kernel, cada um mais adequado a um tipo de problema.
  - Alguns exemplos de Kernel:

Kernels	Equation
1.Linear Kernel Function	$k(x_i, x_j) = 1 + x_i^T x_j$
2.Polynomial Kernel Function	$k(x_i, x_j) = (1 + x_i^T x_j)^p$
3.Radial Basis Kernel Function	$k(x_i, x_j) = \exp(-\Upsilon \lVert x_i - x_j \rVert^2)$
4.Exponential radial basis kernel function	$k(x_i, x_j) = \exp(-\frac{\lVert x_i - x_j \rVert}{2\sigma^2})$
5.Gaussian Radial Basis Kernel Function	$k(x_i, x_j) = \exp(-\frac{\lVert x_i - x_j \rVert^2}{2\sigma^2})$
6.Sigmoid Kernel Function	$k(x_i, x_j) = \tanh(k x_i^T x_j - \delta)$

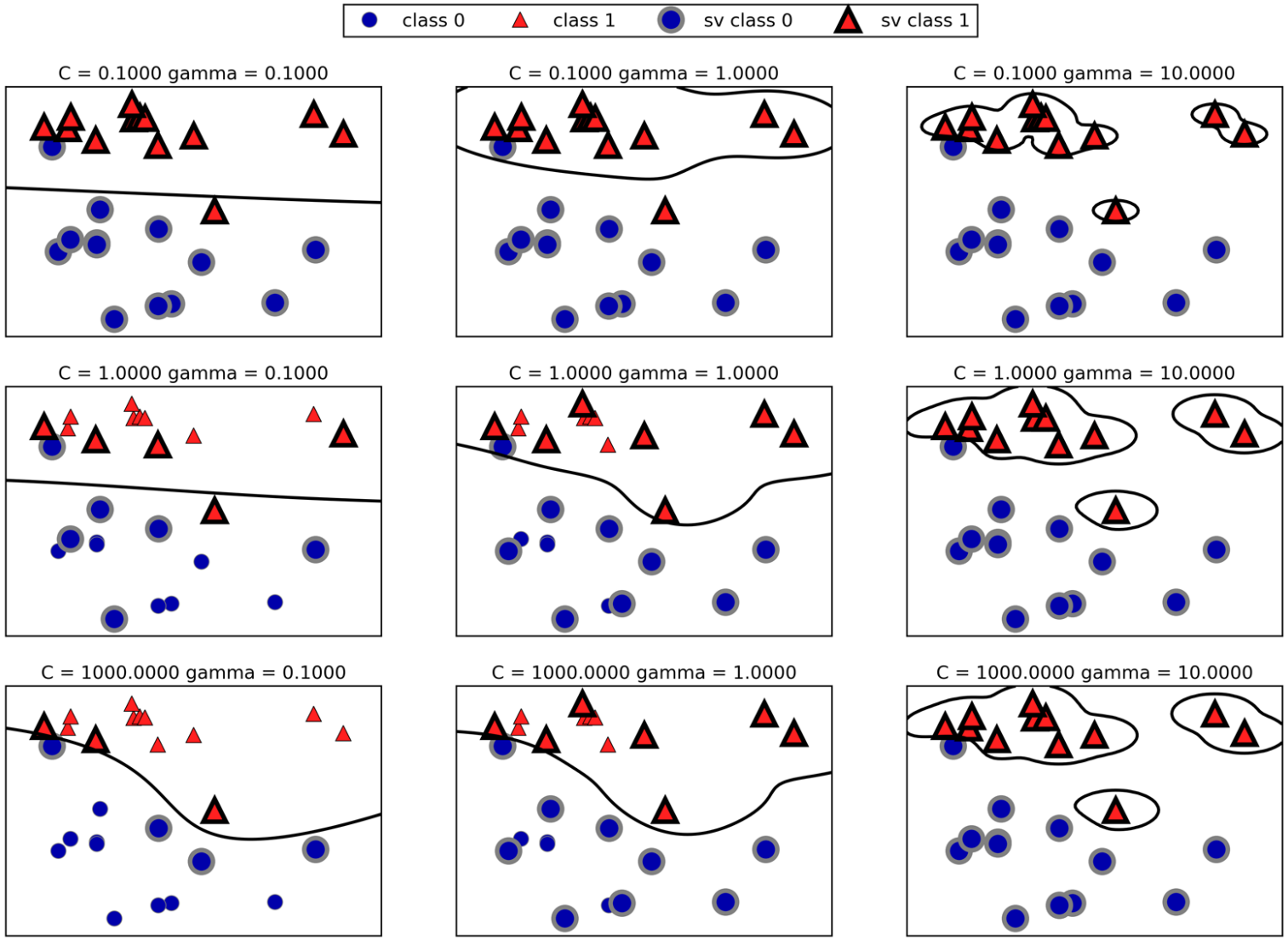
Mais utilizado

  - Em geral, os **cálculos matemáticos** envolvendo kernels são muito complexos.

Support Vector Machines (SVM)

- **Parâmetros de configuração**
  - O SVM possui três parâmetros principais de configuração:
    - **Kernel**: escolher o tipo de kernel que será utilizado.
    - **Gamma**: determina a distância máxima que um elemento pode possuir a ponto de influenciar na construção dos kernels. Na prática, controla o tamanho dos kernels: quanto maior o valor de gamma, menor o raio do kernel. Se o gamma for **muito baixo**, a linha ou curva pode ser **muito suave** e pode acabar não separando bem as coisas. Se o gamma for **muito alto**, a separação pode ser **muito "curvada"** e acabar seguindo muito de perto os pontos de dados.
    - **c**: punição por classificação incorreta. Um valor alto procura fazer a separação com 100% de acerto. Uma valor mais baixo possui mais tolerância a erros, tornando a separação mais suave.

- **Parâmetros de configuração**
  - Influência dos parâmetros:



- **Construindo o script**

  – Primeiramente, o script de pré-processamento deve ser executado, selecionando quais as transformações precisam ser feitas na base.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\PythonWorkspace\2025.1\aula4

...tual\2025-1_Técnicas de Aprendizado de Máquina (reformulação)\aula 3\Programas\pre-processamento.py

pre-processamento.py × credit_data.csv ×

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 31 18:32:59 2025

@author: marco
"""

import pandas as pd
base = pd.read_csv('credit_data.csv')
resumo = base.describe()


# ==========================================================================
#                        Tratando valores inválidos
# ==========================================================================

# Verifica idades inválidas
base.loc[base['Age'] < 0]

# Apagar a coluna
# base.drop('Age', axis=1, inplace=True)

# Apagar apenas os registros com problema
# base.drop(base[base['Age'] < 0].index, inplace=True)

# Preencher os valores com a média
idade_media = base['Age'][base['Age'] > 0].mean().round()
base.loc[base['Age'] < 0, 'Age'] = idade_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, …
checking_maioria	str	6	little
classe	DataFrame	(600, 1)	Column names: Risk
classe_teste	DataFrame	(150, 1)	Column names: Risk
classe_treinamento	DataFrame	(450, 1)	Column names: Risk
cols_classe	list	1	['Risk']
cols_previsores	Index	(18,)	Index: 18 entries, Age …

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024,
15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]:
   ...: """
   ...: Created on Mon Mar 31 18:32:59 2025
   ...:
   ...: @author: marco
   ...: """
   ...:
   ...: import pandas as pd
   ...: base = pd.read_csv('credit_data.csv')
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 3, Col 36 UTF-8 CRLF RW Mem 75%

- **Construindo o script**
  - Para utilizar o SVM, crie um novo arquivo chamado “SVM.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com SVM ##################

# Treinamento
from sklearn.svm import SVC
classificador = SVC(kernel = 'sigmoid', C = 0.9, gamma = 'auto', random_state = 1)
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)

# Resultados na base de treinamento, para verificar overfitting
previsoes_treinamento = classificador.predict(previsores_treinamento)
acuracia_treinamento = accuracy_score(classe_treinamento, previsoes_treinamento)
matriz_treinamento = confusion_matrix(classe_treinamento, previsoes_treinamento)
```

- **Construindo o script**
  - Para utilizar o SVM, crie um novo arquivo chamado “SVM.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com SVM ##################

# Treinamento
```

acuracia_teste	float	1	0.58
matriz_teste	Array of int64	(2, 2)	[[45 25] [38 42]]

```
# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes
matriz_teste = confusion_matrix(classe_teste, previsoes

# Resultados na base de treinamento, para verificar ove
previsoes_treinamento = classificador.predict(previsores_treinamento)
acuracia_treinamento = accuracy_score(classe_treinamento, previsoes_treinamento)
matriz_treinamento = confusion_matrix(classe_treinamento, previsoes_treinamento)
```

Precisamos registrar este resultado para futuras comparações!

- **Registrando os resultados**

  – O arquivo atualizado com os resultados para esta base fica assim:

Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão	
Classe Majoritária	-	-	0.4667	70 80	0 0
KNN	K=5 Distancia Euclidiana	Criação de variáveis dummy	0.5533	41 38	29 42
KNN	K=29 Distancia Euclidiana	Criação de variáveis dummy	0.5867	43 35	27 45
KNN	K=30 Distancia Euclidiana	Criação de variáveis dummy Padronização	0.6333	49 34	21 46
Naive Bayes	-	Apenas LabelEncoder	0.56	38 34	32 46
Naive Bayes	-	Apenas LabelEncoder Padronização	0.5533	38 35	32 45
Naive Bayes	-	Apenas LabelEncoder Sem coluna [‘Duration’]	0.52	39 41	31 39
Árvore de Decisão	criterion='entropy', max_depth=8, random_state=0	Apenas LabelEncoder	0.5734	49 43	21 37
Árvore de Decisão	criterion='entropy', max_depth=6, random_state=0	Apenas LabelEncoder	0.58	43 36	27 44
Árvore de Decisão	criterion='entropy', max_depth=16, random_state=0	Apenas LabelEncoder Sem coluna [‘Duration’]	0.6333	44 29	26 51
Árvore de Decisão	criterion='entropy', max_depth=6, random_state=0	Apenas LabelEncoder Padronização	0.58	43 36	27 44
Random Forest	criterion='gini', max_depth=15, n_estimators=9, max_features=7, random_state=0	Apenas LabelEncoder	0.6533	40 22	30 58
SVM	kernel = 'sigmoid', C = 0.9, gamma = 'auto', random_state = 1	Apenas LabelEncoder Padronização	0.58	45 38	25 42

FIM